Universitat Politècnica de Catalunya

Big Data Management (BDM)
Master in Data Science (MDS)

P2. Trusted, Exploitation and Consumption Zones

Authors:
Joel Delgado Jiménez
Carles Aguilera Pilo

October 11, 2026



Contents

1 Introduction and architecture evolution **2**

2 Instructions **2**

3 Architecture design **3**
3.1 Overview 3
3.2 New services 4

4 Trusted Zone (Silver Layer) **5**
4.1 Camera images. 6

5 Exploitation Zone (Gold Layer) **7**
5.1 Dimensional Modeling 9

6 Data Consumption **10**

7 Data Governance **12**
7.1 Data domains and data products 12
7.2 Implemented mechanism: lineage and quality by construction 12

8 Conclusions **13**

A Dashboard panels **14**

1 Introduction and architecture evolution

This document constitutes the consolidated deliverable for Phase 2 (P2) of our Big Data Management project focused on traffic and urban mobility in Barcelona. In Phase 1 (P1), we implemented the ingestion layer and the Landing Zone architecture, divided into a Temporal Landing Zone (raw files stored in SeaweedFS) and a Persistent Landing Zone (Delta Lake tables built on the same object store). Phase 2 completes the data platform by introducing the remaining layers of the architecture, the Trusted Zone, the Exploitation Zone and the data consumption Zone.

In the P2 design document, we stated our intention to maintain a unified Lakehouse architecture based on SeaweedFS and Delta Lake, avoiding the introduction of specialized databases unless a clear technical requirement emerged. This principle largely guided the final implementation, with two deliberate exceptions discussed in Section 3.

We also anticipated evaluating ClickHouse or a vector database only if project requirements justified their adoption. Ultimately, a columnar analytical engine was not required, as the data volumes involved were moderate and Spark operating on Delta Lake provided sufficient support for analytical transformations. However, we incorporated a vector database (ChromaDB) to support the image-similarity functionality included in the dashboard, a workload that is not well suited to a conventional object-storage-based architecture.

Throughout this document, the terms Bronze, Silver, and Gold are used interchangeably with the Persistent Landing Zone, Trusted Zone, and Exploitation Zone, respectively, reflecting the naming conventions adopted for buckets and DAGs within the project repository.

2 Instructions

The entire platform is containerized and orchestrated, allowing deployment through the same procedure used in P1, with the addition of several new services. Docker must be installed before proceeding. Since the platform includes resource-intensive components such as Airflow, SeaweedFS, Kafka, Spark Master and Worker, ChromaDB, Redis, the Streamlit dashboard, and the synthetic sensor simulators, it is recommended to close other resource-intensive applications beforehand to ensure sufficient system resources during deployment. The complete deployment procedure is outlined below.

1. Clone the project repository and navigate to its root directory:

   ```
   git clone https://github.com/carles-aguilera-pilo/BDM.git
   cd BDM
   ```

2. Build the custom Airflow image, which includes `pyspark`, the Spark provider, and the S3A/Delta JARs required by the Silver and Gold processing jobs:

   ```
   docker compose build
   ```

3. Create the environment configuration file and start the platform services:

   ```
   cp env.example .env
   docker compose up -d
   ```

Execution order. The orchestration is fully driven by Airflow at localhost:8080. One-time and historical loads are launched by manually triggering the `manual_ingestion` DAG, which fans out to the four bootstrap ingestion DAGs: the tram-segment reference table, the rain-prediction dataset, the historical tram-congestion and Guàrdia Urbana accident datasets. The remaining ingestion DAGs run on their own schedules. As in P1, each ingestion DAG triggers its corresponding Bronze-stage pipeline from the Temporal Landing Zone to the Persistent Landing Zone through a `TriggerDagRunOperator`.

The main P2 addition is the `trusted_zone_pipeline` DAG, scheduled hourly, which on successful completion automatically triggers the `exploitation_zone_pipeline`. Both DAGs can also be run manually to force an immediate end-to-end execution.

Streaming services. Three services run continuously outside Airflow, as they model real-time sources rather than scheduled jobs. The synthetic producers `parking-sensor` and `sensor-bcn` push events to Kafka, parking-bay occupancy and per-location traffic flow/speed, respectively, while the `spark-streaming` job consumes these topics and lands them as Parquet. The CLIP image embeddings consumed by the camera-search widget are produced by the on-demand `camera-embedder` service.

Where to look. Every service exposes a UI for inspection:

Service	Endpoint	Purpose
Airflow	localhost:8080	Orchestration: trigger and monitor every DAG
Streamlit	localhost:8501	Consumption dashboard (maps, models, image search)
Kafka UI	localhost:8081	Inspect streaming topics and live events
SeaweedFS Filer	localhost:8888	Browse the four S3 buckets

The four buckets (`landing-zone`, `persistent-landing-zone`, `trusted-zone`, `exploitation-zone`) are served over the S3 API at `:8333` and map one-to-one onto the medallion stages. The codebase mirrors the same structure: `dags/ingestion` and `dags/bronze` hold the landing-stage workflows, `trusted-zone/` the Silver-layer transformations, `exploitation-zone/` the Gold-layer transformations, and `presentation-layer/` the consumption components.

3 Architecture design

3.1 Overview

The architecture design for this second part of the project has followed the foundations established in the first deliverable: We followed the first of the two options proposed in the guidelines, which is extending the existing object store rather than migrating each zone into a dedicated engine. This decision was determined by a set of factors: On one hand, the data lakehouse architecture using the same object-storage for all zones was enough to satisfy our data access requirements, so adding more specific databases increased the management complexity without providing any noticeable benefit. On the other hand, we were constrained by hardware limitations and, since we already were experiencing performance issues managing all Airflow dags and Spark jobs, we avoided introducing new services unless it was necessary.

As will be explained in the following pages, every zone is a separate SeaweedFS bucket, and all tabular data is persisted as Delta Lake tables. Using Delta Lake tables allowed us to easily apply the same functions to read/write tabular data and perform the transformations easily, while preserving all the additional benefits of ACID transactions, time-travel capabilities, etc. Moreover, keeping a single storage technology for all data simplified the integration work of the different datasources.

The following diagram shows an overview of the final system architecture:

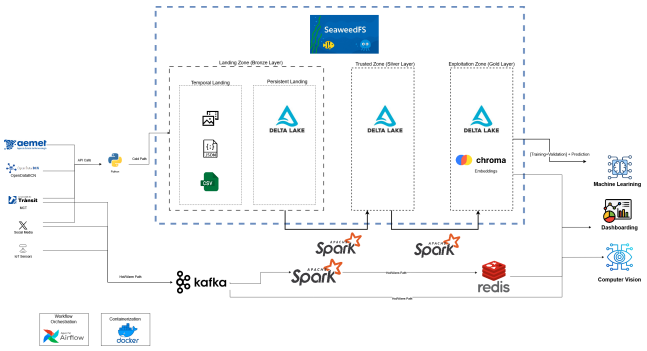


Figure 1: Architecture of The project

3.2 New services

Although we aimed to minimize the addition of new services, there have been components that have been included or re-used in this second deliverable in order to achieve different purposes:

- **Spark (+ Streams)**: for big data processing we use a Spark cluster for distributed and parallel computing. All transformations between the persistent landing zone and the exploitation zone are performed using Spark[1]. Moreover, for the hot and warm path, we reused the Spark cluster to implement Spark streams that process data on-the-fly.
- **ChromaDB (vector database)** stores the CLIP embeddings of the traffic-camera images. The "find the most likely camera location" feature is a nearest-neighbour query in a 512-dimensional space. Resolving it by scanning Parquet would require a full linear pass with a manual distance computation on every read. A vector store performs this similarity search natively and with proper indexing, which is precisely the case the guidelines flag as justifying a dedicated technology. We chose ChromaDB over Milvus for its lightweight, embeddable deployment, which suits our single-node setup without the operational overhead of a heavier vector engine.
- **Redis (key–value store)** holds the live parking counters for the hot path. The dashboard requires sub-second reads of a small set of continuously-refreshed values, an access pattern for which an object store is unsuited but a key–value store is ideal. Fortunately, we noticed that the current Airflow deployment already required a Redis instance running as its Celery broker (on a dedicated database index), so we reused this component by creating a new store for our hot path workloads without adding a new container to the stack.

The Trusted and Exploitation steps run on **Apache Spark**, a small standalone master/worker submitted from Airflow via the `SparkSubmitOperator`, since this is where the substantial joins explodes and aggregations take place. Streaming data from Kafka is handled by a dedicated **Spark Structured Streaming** job. Airflow orchestrates the entire batch chain and Docker containerizes every service, extending the setup established in P1.

Each zone is a SeaweedFS bucket holding Delta tables; Spark performs the Silver and Gold transformations; ChromaDB and Redis serve the two access patterns the object store cannot do. Kafka feeds the warm and hot paths consumed directly by the dashboard, and is also archived to the landing zone by a Spark Structured Streaming job.

The eight data sources carried over from P1 are summarized in Table 1; the classification (structured / semi-structured / unstructured and batch / semi-streaming / streaming) drives the per-type decisions in the next two sections.

[1]The lightweight data processing from the temporal landing zone to the persistent landing zone could also be done with Spark, but we left it after discussing with the teacher because that was evaluated in the first deliverable

Dataset	Source	Flow	Type
Weather predictions	AEMET API	Batch (6h)	Semi-structured (JSON)
Transit posts	Nitter / @transit	Semi-streaming	Unstructured (text)
Traffic cameras	Gencat SCT	Semi-streaming	Unstructured (images)
Traffic incidents	Gencat SCT	Semi-streaming	Semi-structured (XML)
Segment speeds	TomTom API	Batch / semi-str.	Semi-structured (CSV)
Relació trams (geometry)	Open Data BCN	Batch (once)	Structured
Tram congestion	Open Data BCN	Batch / semi-str.	Structured
Accidents Guàrdia Urbana	Open Data BCN	Batch (yearly)	Structured
Parking sensors	Synthetic	Streaming	Structured
Traffic sensors (BCN)	Synthetic	Streaming	Structured
Rainfall dataset	Kaggle / HuggingFace	Batch (once)	Structured

Table 1: Data sources inherited from P1 and their flow / type classification.

4 Trusted Zone (Silver Layer)

The role of the Trusted Zone is to correct representational issues without committing to any particular downstream use. We followed the heuristic from the guidelines: transformations here are information-preserving, such as correcting a type, removing an exact duplicate or flattening a nested record, while lossy, task-specific operations such as imputation, outlier removal and feature scaling are deferred to the consumption layer. The only operation that does not fit this rule cleanly is the enhancement of the camera images, a deliberate exception we justify at the end of the section.

Storage and shipping. The Silver output lives in the `trusted-zone` bucket as one Delta table per dataset, preserving the same source/dataset folder hierarchy as the landing zone so that every table can be traced back to its origin. The step is implemented as the `trusted_zone_pipeline` DAG, which runs `trusted-zone/trusted_zone.py` on Spark. The job walks the Persistent Landing bucket and discovers every Delta root through a lightweight recursive scan that skips the `_delta_log` directory and recognizes Hive-style partition folders. It then routes each root to the appropriate pipeline and writes the cleaned table back to `trusted-zone`. A generic `Cleaner` class performs the transformations common to all datasets, while a small set of per-source pipelines handle the formats that require more specific treatment.

Routing by source. Before any cleaning is applied, the pipeline must determine the appropriate treatment for each source, since not all of them admit the same processing: a structured table such as the accidents dataset requires only generic cleaning, whereas semi-structured sources such as AEMET (nested JSON) or SCT (WFS/GML XML) require a source specific transformation to flatten them into a tabular format. To this end, the `partition_roots_by_pipeline` function classifies each discovered dataset according to its location within the landing hierarchy and routes it to one of four groups: AEMET, SCT, Transit, or a generic group that gathers the remaining structured sources. This way, each data type receives exactly the treatment it requires, rather than imposing a uniform processing that would be inadequate for formats of such different nature.

The generic cleaner, configured through `CleanOptions`, trims string columns, converts empty strings into proper nulls, removes duplicates (either full-row or by a primary-key subset), and sanitizes column names into legal Delta identifiers. None of these operations discards analytical signal. Table 2 details the cleaning applied to each data type.

Type	Dataset(s)	Cleaning applied
Structured	Accidents GU, tram congestion, relació trams, TomTom	Type casting, trim and empty-to-null normalisation, deduplication by primary key where one exists (e.g. `Numero_expedient` for accidents), or full-row otherwise.
Semi-struct.	AEMET forecasts (JSON)	A UDF explodes the nested `prediccion` object into one row per municipality, day and hour. Deduplication is done by (municipality, `elaborado`, day, hour).
Semi-struct.	SCT incidents (WFS/GML XML)	Namespace-aware parsing each `featureMember` into a flat 13-field row and deduplication by `identificador`.
Unstructured	Transit posts (RSS/JSON)	Each post flattened into a tabular row with title, summary, author and timestamp resolved across alternative fields. HTML stripped to plain text, media URL extracted and deduplication is done by post link.
Unstructured	Traffic-camera images	Integrity check and decode. Light Pillow enhancement of brightness, sharpness and contrast (frame-by-frame for GIFs) and copy into the trusted image gallery.

Table 2: Cleaning tasks applied to each data type in the Trusted Zone.

A unifying decision across the semi-structured and unstructured textual sources is that we converted all of them into a common tabular representation in the Trusted Zone. The motivation is that downstream operations, such as the joins, unions and aggregations of the Exploitation Zone, are considerably simpler and more efficient over flat tables than over nested or document-oriented structures. The strategy is the same in every case: each record, whether an XML node or a JSON object, becomes one row of a tabular schema, and each of its attributes becomes a column, so that the original information is preserved while its shape is regularised. Keeping attribute names aligned with column names also makes it unambiguous which feature each numeric or categorical value refers to.

The AEMET forecast is the most complex datasource in this context: The datasource (that is, the API) returns a deeply nested object in which each day holds arrays of hourly readings for temperature, humidity, precipitation, wind, sky state and others, so a single landing record contains an entire multi-day forecast. We expand it into a table with one row per (municipality, day, hour). This, even though it changes drastically the representation, the original information is kept and that is why it is done as part of the trusted zone. The SCT incidents follow the same principle: the WFS/GML payload is parsed with namespace awareness, and each `featureMember` is exploded into a flat row of thirteen fields. The Transit posts are handled analogously, with each RSS/JSON entry mapped to a single row carrying its title, author, timestamp and cleaned text.

4.1 Camera images.

The image enhancement is the one Trusted-Zone operation that is not strictly information-preserving, so it deserves an explicit justification. By the guideline's own example, brightness and contrast adjustment is a task-specific transformation that usually belongs closer to consumption and exploitation layers. We still apply a mild, fixed enhancement here, for two reasons.

First, its scope is narrow. The enhanced images feed only the CLIP embedding step in the Exploitation Zone, where a normalised input gives more stable vectors. The user-facing assets, the live camera gallery and the YOLO detector, do not use them. They take the warm path: the Kafka event on the `traffic_cameras` topic carries only metadata and a pointer to the *raw* image in the landing bucket (an `s3://` URL), and the dashboard uses that pointer to fetch the original frame from SeaweedFS on demand (Section 6). The enhancement therefore never touches the frames a user actually sees.

Second, the adjustment is small, uniform and identical for every image. It helps the one consumer that uses it and encodes no assumption that could harm another task. This matches the guideline's own test: a transformation is safe when it does not hurt any other downstream task, and here only the

embedding step is involved.

In conclusion, this layer applies only the minimal transformations that do not compromise any future task: basic cleaning, type normalisation and deduplication, leaving any task-specific operation for later stages. The main effort in this zone, however, was not the cleaning itself but the design of tabular schemas for the semi-structured and unstructured sources. We made the deliberate decision to unify all heterogeneous formats (JSON, XML, RSS) into a common tabular representation, rather than keeping them in their original nested structures. The reason is that the operations of the Exploitation Zone, namely joins, unions and aggregations, are considerably simpler and more efficient over flat tables than over nested or document-oriented structures: a consistent tabular schema lets us operate on the data at a lower computational cost and with much clearer logic. Therefore, while structuring these sources was the most demanding part of the Trusted Zone, it is precisely the decision that makes the rest of the pipeline more direct, maintainable and scalable. See the internal strcuture of the Trusted-zone / Silver in the Figure 2

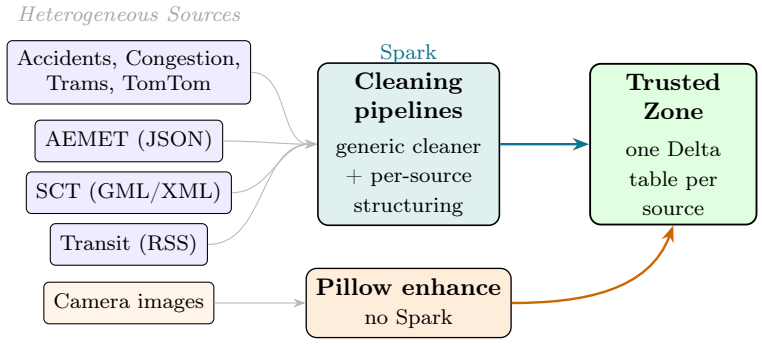


Figure 2: Trusted Zone (Silver layer)

5 Exploitation Zone (Gold Layer)

In the Exploitation Zone we move from correct data to convenient data: tables shaped specifically for the concrete questions the dashboard asks. The design principle we followed is that the exploitation layer should leave the data in the optimal format to be consumed quickly and efficiently by the presentation layer, avoiding any expensive computation at query time. This is a common practice in real-world data architectures: the presentation layer should not have to transform, join or aggregate data on the fly, but simply read it already prepared. We therefore precomputed and materialized every asset that the consumption layer would otherwise have to recompute on each page load, shifting the computational cost to the batch pipeline rather than onto the end user. As the guidelines suggest, we kept the catalog deliberately small and tied each asset to a concrete consumption task, rather than producing derived tables that no downstream task would use.

Storage and shipping. Gold tables are written to the `exploitation-zone` bucket as Delta, while the camera embeddings are stored in ChromaDB. The `exploitation_zone_pipeline` DAG is triggered by the Trusted DAG on success and runs three steps: a `BashOperator` that reorganizes the camera images into a per-site gallery, a second one that pushes the CLIP embeddings of those images into ChromaDB, and the main Spark job `exploitation-zone/exploitation_zone.py`, which builds the tabular assets.

Organization and new assets. The assets we materialize fall into four groups (Table 3): denormalizing joins, aggregated KPIs, ML-ready datasets, and unstructured/semantic outputs.

Asset	Description	Serves
Denormalizing joins		
`trams_congestion_enriched`	Tram congestion joined with the static `relació_trams` attributes on `id_tram`.	Congestion analysis
`trams_map`	One polyline per tram, built from the coordinate vertices, plus its latest congestion state via a window function.	Live map widget
Aggregated KPIs		
`TOMTOM/`	Per measurement point: average current and free-flow speed, speed difference and congestion %.	Speed KPIs
`aemet/climate_by_municipio`	Daily average temperature, feels-like and humidity per municipality.	Climate panel
`accidents_gu_by_district`	Totals of deaths, serious and minor injuries, victims and vehicles per district and neighbourhood.	Accident KPIs
ML-ready datasets (pre-split 80/20)		
`aemet/ml_rain_{train,test}`	Features and binary target `ha_plogut` (whether it rained).	Rain predictor
`accidents_gu_ml_{train,test}`	Features and binary target `accident_greu` (a death or a serious injury).	Severity predictor
Unstructured / semantic outputs		
`traffic_cameras/gallery/`	Camera images reorganized into a per-site gallery.	Camera browser
ChromaDB `camera_images`	CLIP (ViT-B/32) embeddings in cosine space.	Image search
`incidents/daily_incidents`	SCT incidents and Transit posts unioned into one schema-aligned feed.	Incident timeline

Table 3: Data assets generated in the Exploitation Zone, grouped by purpose. Each asset is tied to a concrete consumption task.

A couple of design choices are worth mentioning. First, the train/test split for the two ML datasets is done here, in Spark, with a fixed seed, rather than in the dashboard. The idea is to leave the training and validation sets already materialized and prepared, so that the consumption layer can access them quickly and directly without having to reconstruct them on every run. Reshaping and labelling are reusable and expensive operations, so they belong in the lakehouse; the scaling and model fitting, which are model-specific, belong instead to the presentation layer, and that is where we left them. This establishes a clear boundary: the Exploitation Zone prepares the data ready for training, and the consumption layer only has to fit the model.

Second, `trams_map` is a small example of computing a new asset purely for serving. It pre-builds the geometry as ready-to-draw polylines from the join of `relacio_trams` and `trams_congestion`, attaching the most recent congestion state per tram via a window function (`row_number()` ordered by timestamp, partitioned by `id_tram`) applied over the Silver Delta tables. This produces a Gold table that the cold-path tab of the dashboard can render immediately, without any per-request geometry assembly or aggregation.

However, this cold-path asset presents a problem: the trusted dag executes hourly, so the congestion state it carries is at most as fresh as the last hourly pipeline run, which is too stale for a live traffic map. For this reason we also implemented a warm-path version of the same view. A background thread in the dashboard consumes the `trams_congestion` Kafka topic and maintains an in-memory dictionary of the latest record per `id_tram`. At render time, the widget loads the static geometry from the Gold `trams_map` table (the polylines do not change and are correct to cache), and performs a lightweight in-memory `merge` between that geometry and the live Kafka snapshot on `id_tram`. The result is the same map, but with congestion levels refreshed every two minutes rather than every hour.

Third, the `daily_incidents` feed is a genuine integration of two very different sources (roadworks in XML and posts in RSS) into a single timeline. We compute the union of both sources here, in the Exploitation Zone, so that the information is already consolidated and ready to be accessed and displayed immediately. The presentation layer therefore performs no join or schema reconciliation at query time: it only reads the already-integrated feed and shows it as a single list.

5.1 Dimensional Modeling

The join and aggregation operations described in the previous section are not arbitrary; they follow a specific dimensional model. In this section we justify those operations through the concept of the *star schema*, which lets us explain why we structured the data the way we did.

A star schema is a multidimensional modeling design, widely used in OLAP systems, in which a central fact table (holding quantitative measures such as counts or averages) connects to several dimension tables (holding the descriptive attributes along which those measures are analysed, such as time, location or category). This organization optimizes precisely the analytical queries, based on aggregations and filtering, that feed the analytical views of the dashboard.

Adopting this model is especially relevant in our case, since the final presentation layer is an exploratory analysis tool rather than an operational or transactional system. Whereas a transactional system (OLTP) would prioritise normalization to avoid update anomalies, an analytical workload such as ours benefits from the pre-joined, denormalized structure of the star schema, which reduces query complexity and latency at consumption time.

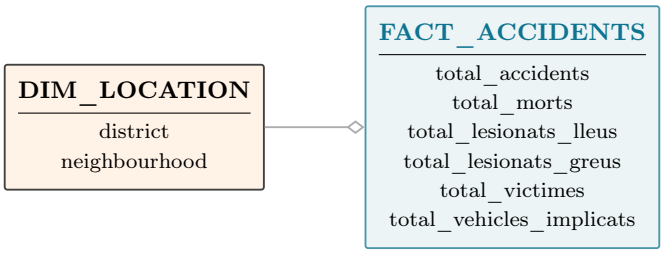


Figure 3: Star schema 1 — Accidents.

This Figure 3 shows the multidimensional design of the accidents fact table. The central measures are obtained by aggregating the Guàrdia Urbana accident records by district and neighbourhood, so that for each area we have the totals of accidents, deaths, injuries (minor and serious), victims and vehicles involved. The dimension that structures these measures is therefore the geographic location (`DIM_LOCATION`), with its natural district → neighbourhood hierarchy. This organization lets the dashboard answer analytical questions such as "which areas concentrate the most serious accidents" immediately, without having to scan the individual records at query time.

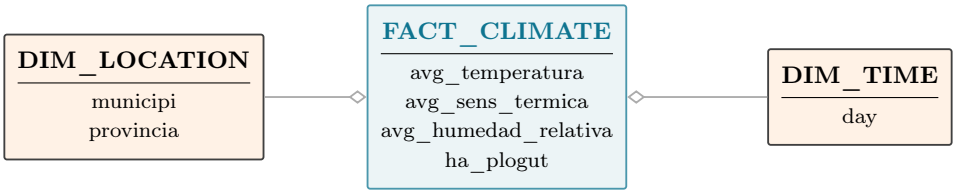


Figure 4: Star schema 2 — Climate.

In this case, the climate design follows the same pattern as the previous one, but with two key differences in its dimensional structure. First, the geographic granularity is finer: we aggregate by municipality and province (`DIM_LOCATION`) instead of by district and neighbourhood. Second, and more

importantly, a second dimension appears, time (`DIM_TIME`), at a daily granularity. From the AEMET source, for each combination of municipality and day we compute the metrics of the fact table: the average temperature, feels-like temperature and humidity, together with a binary indicator of whether it rained (`ha_plogut`). The result is a two-dimensional star schema that allows these climate metrics to be analysed both in space (by municipality) and in time (day by day), which is precisely the query that feeds the climate panel of the dashboard.

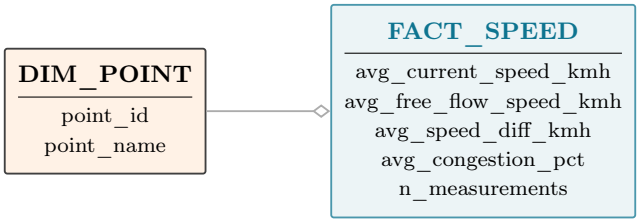


Figure 5: Star schema 3 — Traffic speed.

Finally, the last star schema models traffic speed by measurement point in Barcelona, from the information received from TomTom. Unlike the previous cases, here the aggregation is performed over a single table, with no join required: we group the readings by measurement point (`DIM_POINT`, identified by `point_id` and `point_name`) and compute the traffic metrics. These include the average current speed, the average free-flow speed, their difference and the average congestion percentage, together with the number of aggregated readings (`n_measurements`). The result is a fact table that summarises the congestion state of each point in the road network, ready for the dashboard to display the speed indicators without recomputing them on every query.

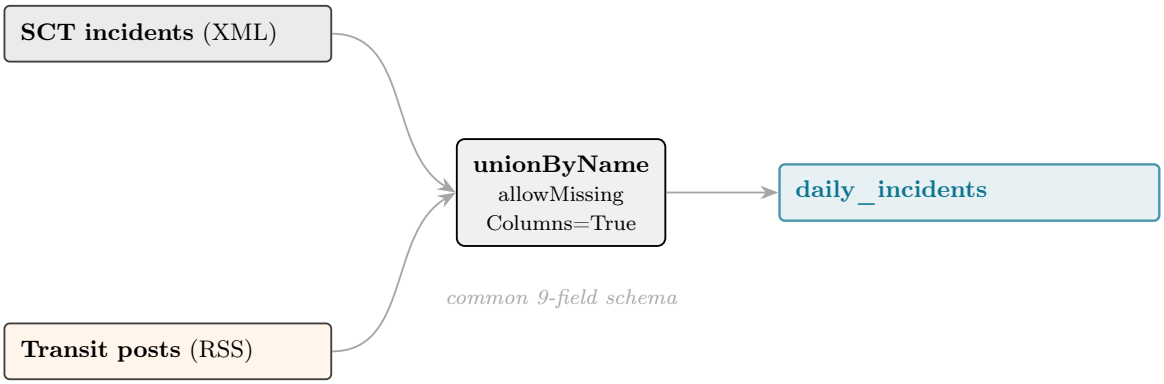


Figure 6: Union of the two incident sources into `daily_incidents`.

The `daily_incidents` table integrates two incident sources of different nature: the SCT incidents and roadworks (XML) and the transit account posts (RSS). Since both describe the same type of entity, we project them onto a common nine-field schema, filling with `NULL` the attributes that one source has but the other does not (for example, `carretera` only exists in SCT and `autor` only in the posts). Once the schemas are aligned, we merge them with `unionByName(allowMissingColumns=True)`. The result is a single timeline of incidents that the dashboard shows as a unified list, without reconciling the sources at query time.

6 Data Consumption

All consumption happens through a single Streamlit dashboard, connected to the three latency tiers the guidelines ask about: the cold path (batch), the warm path (micro-batch) and the hot path (real-time).

Cold path. This is the batch chain described above. The dashboard reads the Exploitation Zone Delta tables (tram map, TomTom KPIs, climate, accidents and incidents) with a few-minutes cache that is refreshed every time the hourly pipeline runs. It is the right path for aggregated and reference data, which do not need constant updating.

Warm path. For the data that benefits from being fresher, the dashboard skips the lakehouse and consumes it directly from Kafka. Background threads read the `trams_congestion` and Sensors-IoT topics into in-memory dictionaries and redraw a pydeck map roughly every two minutes.

Traffic cameras (warm path). The cameras follow the same warm path, but with a deliberate design split between metadata and binary content. For every camera it polls, the ingestion DAG writes the raw image to SeaweedFS and publishes a small JSON event to the `traffic_cameras` topic carrying only the metadata (road, municipality, timestamp and an `s3://` pointer to the image). A background consumer keeps an in-memory index of the latest events per camera; when a camera is displayed, the widget reads the metadata from the index and fetches the image bytes on demand from SeaweedFS. This way, the lightweight, fast-changing metadata flows through Kafka, while the heavy binary payload stays in the object store and is pulled only for the frame the user is looking at.

Hot path. The parking counter is the minimum-latency path. A thread consumes the `parking-lot-events` topic, keeps a running free/occupied count in Redis, and a Streamlit fragment refreshes it every five seconds. To make the difference between the three tiers visible, the same tram map is shown in three tabs (warm via Kafka, cold via batch, and IoT sensors); the static tram geometry is loaded once from the cold store, since it is correct to treat it as reference data.

A more specific list of services grouped by theme is detailed below:

Real-time monitoring (live maps and counters).

- **Real-time parking.** A free/occupied counter fed by Kafka, aggregated in Redis and refreshed every 5 seconds (hot path).
- **Live tram traffic state.** An interactive pydeck map of the congestion state of each tram segment, consumed live from Kafka and refreshed every 2 minutes (warm path).
- **IoT traffic sensors.** A point map of the synthetic sensors deployed across the city, colour-coded by congestion level (low/medium/high) and average speed, updated continuously from Kafka (warm path).
- **Real-time incidents and posts.** The `daily_incidents` feed, combining SCT incidents and roadworks with the transit account's social-media posts into a single daily list.

Analytical visualizations (curated data).

- **Speed KPIs (TomTom).** Average current speed, free-flow speed and congestion percentage per measurement point.
- **Climate (AEMET).** Charts of the evolution of temperature, feels-like temperature and humidity per municipality.
- **Accidents (Guàrdia Urbana).** Totals of accidents, victims and injuries per district and neighbourhood.

Machine-learning and vision tasks.

- **Rain prediction.** A logistic regression trained on page load from `ml_rain_{train,test}`, reporting accuracy, precision, recall and F1, a confusion matrix and an interactive "what-if" widget.
- **Accident-severity prediction.** A class-balanced logistic regression on `accidents_gu_ml_{train,test}`, with the same set of metrics and interactive prediction. These two tasks realize the forecasting service promised in P1, using aggregated structured data as input.
- **Vehicle detection (YOLOv8).** The model runs live on a selected camera frame and counts vehicles by class (car, bus, truck, motorcycle, bicycle), always on the freshest snapshot retrieved through the warm path rather than on a batch materialized gallery.
- **Semantic image search (CLIP + ChromaDB).** The user uploads a photo, we embed it with CLIP and query ChromaDB for the nearest camera location in the vector space. Here we redirected the initial P2 design: we had planned tweet-topic classification by embedding similarity, but found the camera-localization use case more compelling and better aligned with the mobility theme, so the embedding and similarity machinery went there instead.

To conclude this section, we refer the reader to Appendix A, which collects screenshots of the final interface. While the same result can be reproduced by following the instructions in Section 2, we provide these images so that the evaluators can quickly and clearly inspect the complete dashboard and all the components described above.

7 Data Governance

Although governance is optional, as the pipeline grows in zones, sources and technologies, it becomes harder to manage, which is precisely what data governance addresses. We treat the Exploitation Zone not as loose tables but as *data products* grouped into *data domains*, and implement one governance mechanism end to end, following the three points suggested in the guidelines.

7.1 Data domains and data products

Reflecting on our business context, monitoring and anticipating mobility in Barcelona, the Exploitation-Zone assets group into three data domains, which lets us attach standards, ownership and policies at the domain level rather than per table:

- **Traffic & congestion.** `trams_map`, `trams_congestion_enriched` and the TomTom speed KPIs.
- **Safety & incidents.** The accident-severity dataset (`accidents_gu_ml`), the accidents-by-district KPIs and the `daily_incidents` feed.
- **Environment & vision.** The rain-prediction dataset (`ml_rain`), the climate KPIs and the camera vision products (YOLO detections and CLIP embeddings).

This perspective reaffirmed the organization we had already built around the consumption tasks, but prompted one refinement: the camera-derived assets, initially two separate outputs (YOLO detections and CLIP embeddings), are more naturally governed as a single Environment & Vision family, since they share source, ownership and access rules.

7.2 Implemented mechanism: lineage and quality by construction

We focus on **lineage tracking** and **data-quality validation**, since for an analytics platform data quality is the priority concern. Both are enforced structurally by the architecture, without an external catalog or validation engine.

Lineage. The medallion design makes provenance explicit: since each zone preserves the same folder hierarchy, any Exploitation table traces deterministically back to its source. The Delta `_delta_log` records the full write history (enabling time-travel and auditing), and the Airflow DAG graph documents task-level lineage (ingestion → bronze → trusted → exploitation).

Quality. Checks are embedded at the zone boundaries: the Trusted Zone enforces PK deduplication, type casting, empty-to-null normalisation and Delta schema enforcement; the Exploitation Zone drops rows with missing predictors and ensures both target classes are present before producing a model-ready table.

Benefits and trade-offs. Enforcing governance structurally means traceability and reliability require no additional system: they follow from the medallion design. Each zone is a separate checkpoint, so failures are easy to locate and trace, and malformed records are filtered at each boundary before reaching the models or dashboard. The cost is storing the data three times and the latency of writing each zone before the next, which is minor at our scale and outweighed by the traceability gained. At a larger scale it could be extended with a data catalog (DCAT, Apache Atlas) and a validation engine to externalise the quality rules we now encode in code.

8 Conclusions

This project combines a high number of different datasources with different update frequencies and orchestrates their processing to transform data so that it can be consumed and provide meaningful value. By using different technologies, we have developed an architecture that supports 3 main types of data according to their freshness (cold, warm and hot paths), implemented their transformations and programmed a set of services that consume this data to show an example of how it can be used.

During this project we have encountered some limitations and difficulties. Two are most important: First, the high number of datasources required a careful examination of how to organize them and which transformations apply. At times, it was very easy to get lost through the transformations implemented with Spark. Secondly, and most importantly, the hardware constraints combined with the deployment of expensive containers (Airflow and Spark) and some dags being executed at similar points in time made it very difficult to evaluate and troubleshoot all services at the same time. Often, we had to shut down some workloads in order to be able to comfortably debug others. This has to be considered when desiring to deploy this stack.

Regarding the future steps, we acknowledge we could improve some quality aspects of the provided interface and the services. However, in this project we have prioritized elaborating the data management pipeline and a prototype of possible services as the emphasis throughout was on the data management and the flow between zones rather than on user experience or squeezing accuracy out of the machine learning models. Also, more complex integrations of some datasources could be applied, given that they have not been implemented due to lack of time (an example of this could be integrating tom tom data with trams by performing a join on euclidean distance based on latitude and longitud).

Nevertheless, we consider this project has been a great oportunity to experience the complexities of data engineering involving distributed computing, data integration and organization and other challenges that arise when managing data at scale.

A Dashboard panels

This appendix presents the final dashboard interface, providing visual evidence of each consumption task described in Section 6. The screenshots are organized following the same structure as the report: the real-time monitoring views (parking, live tram state, IoT sensors and the incidents feed), the analytical visualizations (speed, climate and accident KPIs), and the machine-learning and computer vision tasks (rain and accident-severity prediction, vehicle detection and semantic image search). Together, they illustrate how the curated data from the Exploitation Zone is ultimately served to the end user in the presentation layer.

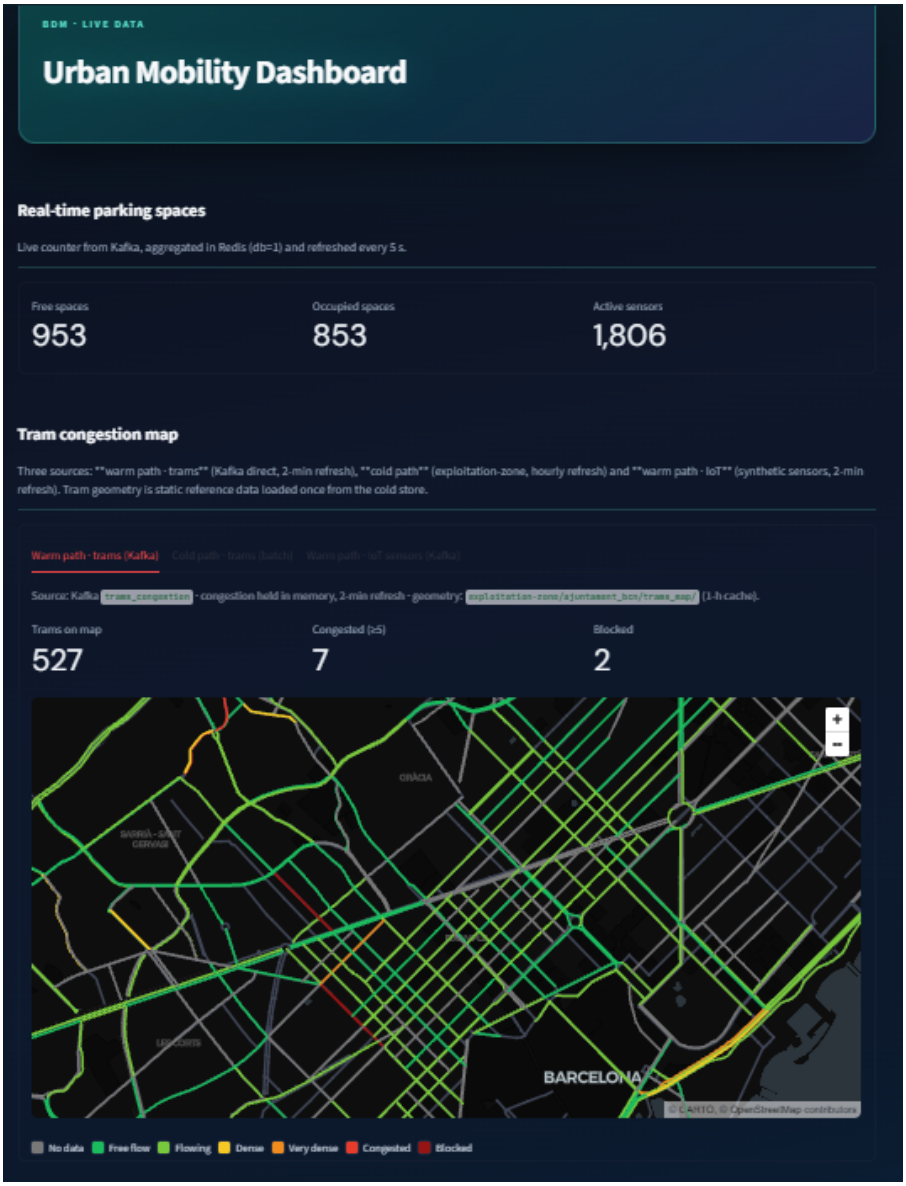


Figure 7: Live traffic state and IoT sensors

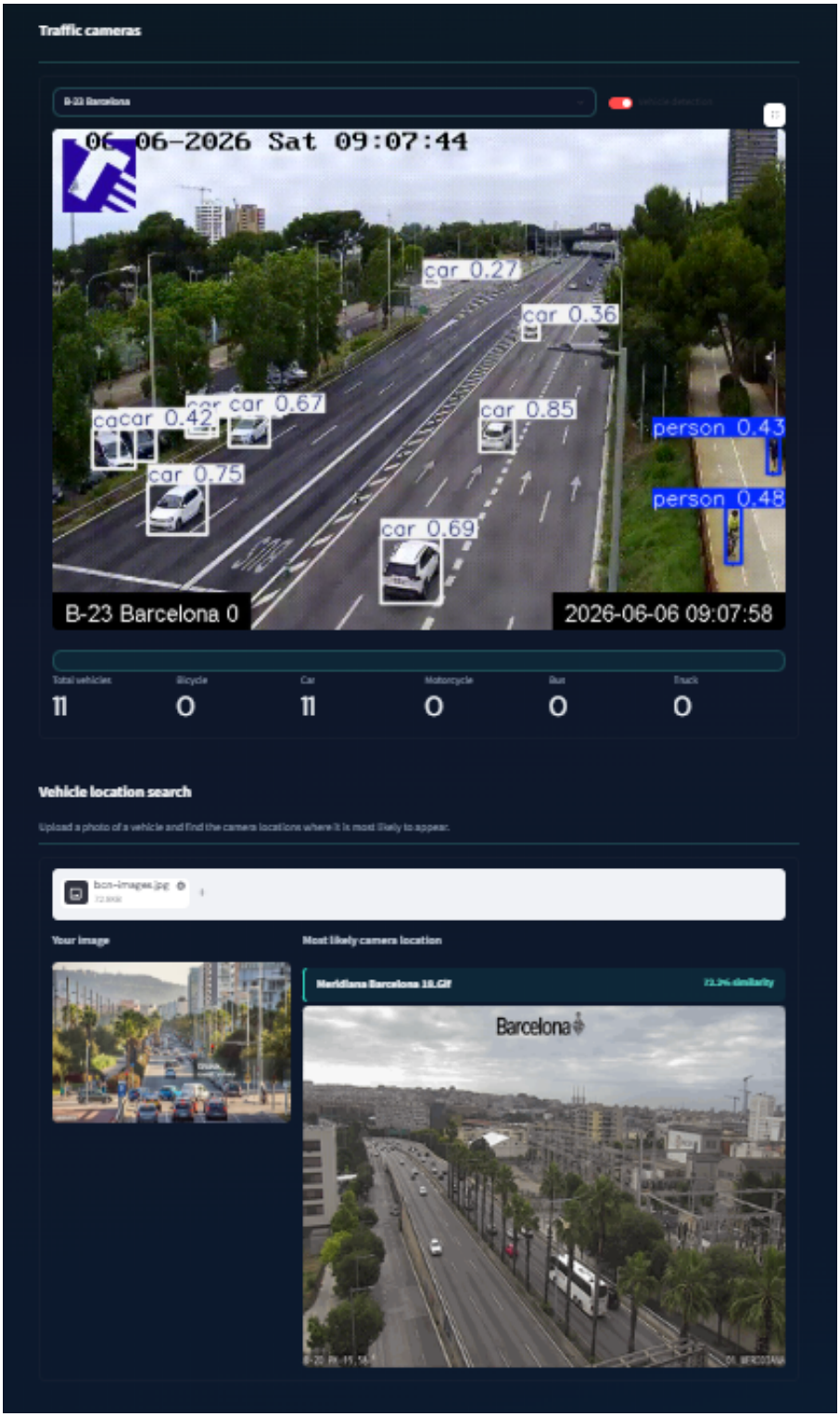


Figure 8: Live camera tasks: vehicle detection (YOLO) and image search (CLIP)

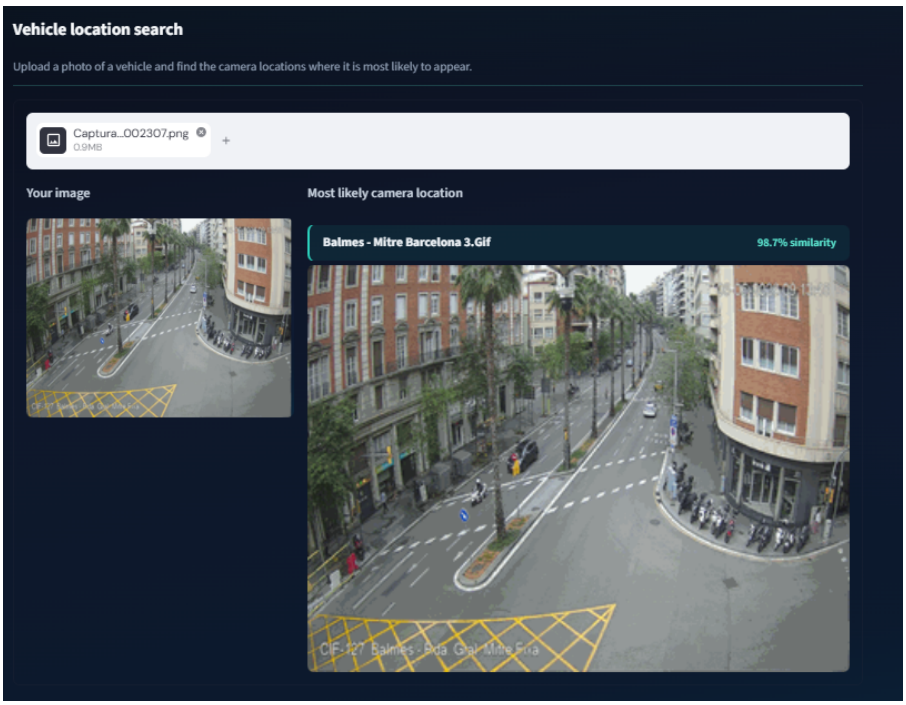


Figure 9: Image similarity search over camera frames



Figure 10: Rain prediction model and AEMET climate dashboard



Figure 11: Accident analytics and severity prediction (Guàrdia Urbana)

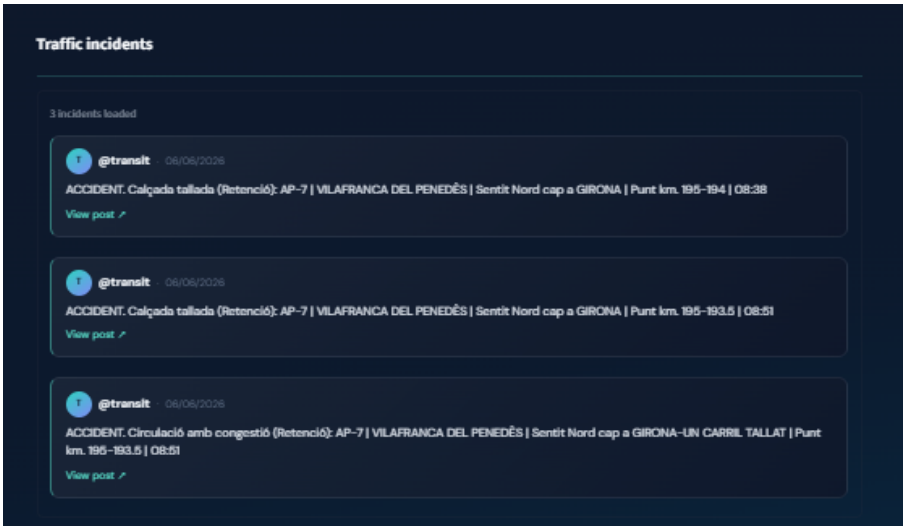


Figure 12: Unified incidents feed: SCT and @transit posts

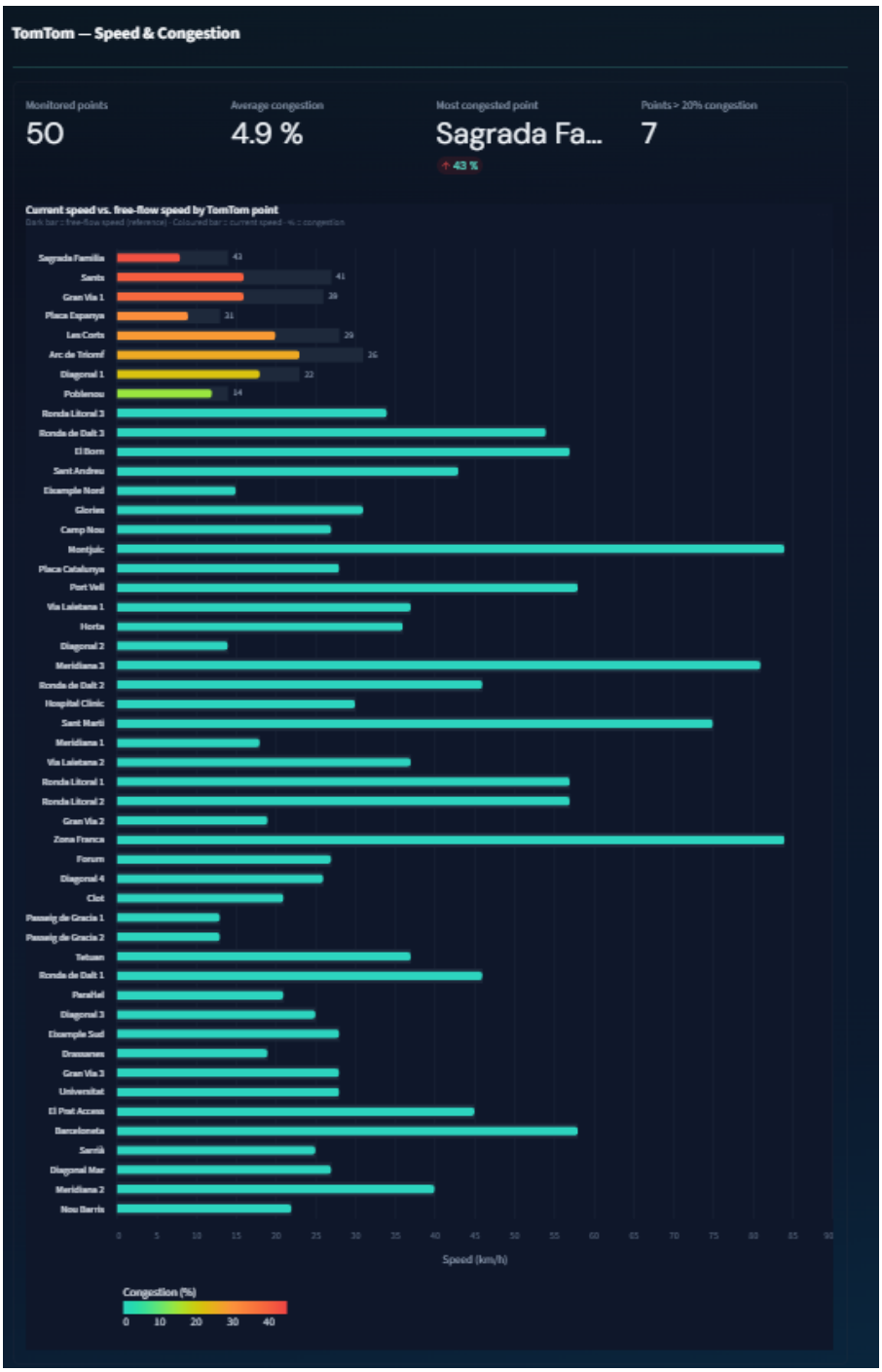


Figure 13: Traffic speed analysis (TomTom)